

This is **excellent**. In fact, you've already done something that most projects never do: you've separated **documentation from code**.

Looking at your folder structure, I have only one recommendation:

Treat this repository as if you are writing a book, not documentation.

You're not documenting code—you are documenting an entire domain.

The Next Step

Don't touch the backend.

Don't touch React.

Don't touch the database.

Instead, let's start writing the **foundation**.

There are exactly **five documents** that everything else depends on.

Stage 1 — The Foundation

These five documents define the soul of the product.

```
docs/  
  
00-overview/  
  
README.md  
  
Vision.md  
  
Mission.md  
  
Goals.md  
  
Glossary.md
```



These answer:

- Why does this product exist?
- Who is it for?
- What problem does it solve?

- What words mean what?

Stage 2 — Philosophy

You already have this folder.

I'd expand it.

```
01-philosophy/
```



```
WhyThisExists.md
```

```
KnowledgeFirst.md
```

```
PoetryAsKnowledge.md
```

```
HumanExperience.md
```

```
CrossCivilization.md
```

```
EditorialPrinciples.md
```

```
ReadingPhilosophy.md
```

Notice that these are **essays**, not technical docs.

They tell every future contributor how to think.

Stage 3 — The Domain

This is where the real work begins.

Instead of coding, we're going to define the universe.

```
03-domain/
```



```
Poem.md
```

```
Verse.md
```

```
Poet.md
```

```
Theme.md
```

```
Symbol.md
```

Concept.md

Motif.md

Language.md

Translation.md

Commentary.md

Each file answers one question.

Example:

Theme.md

What is a Theme?

Definition

Examples

Relationships

Hierarchy

Examples from literature

Examples from Ghalib

Database implications

Future AI implications



That single document will influence

- database
- search
- recommendations
- UI

Everything.

Then

Ontology.

This becomes the company's intellectual property.

04-ontology/

HumanExperiences.md

Themes.md

Symbols.md

KnowledgeGraph.md

RelationshipTypes.md

Taxonomy.md



This folder will probably become your largest folder.

Then

The database.

Notice something.

We don't create

Poems table



Instead

we create

Poem.md



Only after that

Table_Poems.md



Because

Concepts come before implementation.

I also think you're missing one folder

This would be my addition.



Inside

AnalyzeQuran.md



Rekhta.md

Genius.md

Wikipedia.md

StanfordEncyclopedia.md

LessonsLearned.md

Every time you study another product

you document it.

Not

"This app is nice."

Instead

Feature



Word Explorer

What they do well

Every word is clickable.

Weakness

No philosophical commentary.

Our version

Each word also links to

History

Mysticism

Psychology

Cross references

This prevents forgetting good ideas.

Another folder I'd create

```
22-decisions/
```



Every important decision gets documented.

Example

```
ADR-001
```



```
Why PostgreSQL
```

```
Instead of Neo4j.
```

Another

```
ADR-002
```



```
Why we use Themes instead of Categories.
```

Five years later

you'll know why.

Finally, here's the roadmap I'd follow

Month 1

No coding.

Only docs.

Seriously.

```
Vision
```



```
Mission
```

```
Philosophy
```

```
Domain
```

Month 2

Database

Knowledge Graph

Search

API



Month 3

Now ask AI

Build backend.

Month 4

Now ask AI

Build frontend.

Month 5

Editorial CMS.

Month 6

AI.

One architectural change I'd make

I noticed your folders are organized mostly by **software concerns** (API, backend, frontend, etc.). That's good, but I think the project will be stronger if it's organized first by **knowledge**,

then by software.

For example, I would introduce a dedicated `04-ontology/` that becomes the heart of the repository. This folder would define things like:

- Human Experiences
- Themes
- Symbols
- Motifs
- Literary Devices
- Relationships
- Knowledge Graph Rules

Every other part of the system—the database, APIs, search engine, recommendations, and eventually AI—would derive from this ontology.

In other words:



```
Ontology
  ↓
Domain Model
  ↓
Database
  ↓
API
  ↓
Frontend
  ↓
AI
```

This order ensures that technology serves the intellectual model, not the other way around. For a platform whose core value is literary understanding, I think that's the right foundation.